Chu and Liu *Discover Artificial Intelligence* (2025) 5:147
https://doi.org/10.1007/s44163-025-00381-y

Discover Artificial Intelligence

RESEARCH Open Access

Based on BERT-GPT-GNN converged architecture: intelligent generation engine for complex SQL queries in business intelligence

Shiwei Chu[1*] and Jie Liu[2]

*Correspondence:
Shiwei Chu
chushiwei2022@bjfu.edu.cn
[1]School of Electronic Information Engineering, Anhui University, Hefei 230088, Anhui, China
[2]iFLYTEK Co., Ltd, Hefei 230088, Anhui, China

Abstract
Modern enterprises increasingly rely on data-driven decision-making, but traditional SQL (Structured Query Language) queries require professional knowledge, limiting their use by non-technical personnel. With the advancement of natural language processing technology, especially the application of deep learning generative models, text2SQL (Text to SQL) conversion has become possible. RAG (Retrieval-augmented Generation) further improves the accuracy and relevance of answers by combining the advantages of retrieval and generation. This article aims to develop a text2SQL business intelligence system based on RAG, which enables enterprise users to seamlessly extract actionable insights from complex databases via intuitive natural language queries, streamline data retrieval processes, lower technical barriers for non-specialist users, and achieve state-of-the-art performance in SQL query generation for complex tasks. Using BERT (Bidirectional Encoder Representations from Transformers) model for vectorized retrieval and GPT-4 (Generative Pre-trained Transformer 4) pre-trained model for generation, combined with GNN (Graph Neural Network) modeling database structure, the ability to generate complex queries is improved, and the semantic understanding ability of the model is iteratively optimized through user interaction and feedback mechanism. The experimental results show that BERT + GPT-4 + GNN performs excellently in matching accuracy for multi-table joins and nested queries. The query matching accuracy of multi-table joins of BERT + GPT-4 + GNN is 52.3% and 55.1%, respectively, when the beam width is 1 and 10. The query matching accuracy of nested queries with multi-table joins of BERT + GPT-4 + GNN is 60.2% and 61.9%, respectively, when the beam width is 1 and 10. The user satisfaction score of BERT + GPT-4 + GNN is the highest, which verifies its superiority in practical applications. The text2SQL business intelligence system based on RAG proposed in this article significantly improves the ability to process complex queries and reduces data access barriers, thereby providing enterprise users with more convenient and efficient database query tools.

Keywords Business intelligence systems, Natural language big models, Retrieval-augmented generation, Text to structured query language, Natural language queries

© The Author(s) 2025. **Open Access** This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by-nc-nd/4.0/.

1 Introduction

With the evolution of the e-commerce landscape, modern enterprises increasingly rely on data-driven decision-making to enhance competitiveness and optimize operations. For e-commerce enterprises [1, 2], it is crucial to understand market trends and consumer behavior, and to maximize the utilization of inventory and marketing strategies through the analysis of product sales data. Traditional SQL queries are complex and difficult for non-technical personnel to use. Text2SQL technology facilitates data access by converting natural language into SQL queries, thanks to advancements in natural language processing technology, especially deep learning generative models [3, 4]. However, simple generative models may struggle with complex problems. By combining retrieval and generation, the RAG (Retrieval-augmented Generation) model [5, 6] can improve the accuracy and relevance of answers. Our research aims to develop a Text2SQL business intelligence system based on the RAG framework, enabling enterprise users to seamlessly extract actionable insights from complex databases via intuitive natural language queries, streamlining data retrieval processes, lowering technical barriers for non-specialist users, and achieving state-of-the-art performance in SQL query generation for complex tasks [7].

By reducing the technical barriers for non-technical personnel to use databases, this research promotes the popularization of data-driven decision-making, which has significant practical importance. Our model innovatively combines BERT's contextual understanding ability, GPT-4's generative advantages, and GNN's graph structure modeling capabilities to improve the accuracy of SQL query generation through multimodal fusion. Compared to previous work, our model uses BERT to capture deep semantics, addressing the difficulty of docking natural language with database models; GPT-4 provides smoother SQL generation in response to the limitations of inconsistent context generation; and GNN enhances the modeling of inter-table relationships, overcoming the bottleneck of grammatical structure reasoning under complex database models.

This paper is organized as follows: Sect. 1 introduces the background and research motivation of business intelligence systems, and explains the objectives and significance of this study; Sect. 2 reviews the research progress of related work and analyzes the advantages and disadvantages of existing methods; Sect. 3 describes in detail the methods of building business intelligence systems, including data collection, analysis framework and technical implementation; Sect. 4 shows the experimental results, evaluates the performance of the system and the actual application effect; Finally, Sect. 5 summarizes the research results and points out the direction and potential improvements for future research.

2 Related work

Business intelligence systems have long supported enterprise decision-making through data analysis and information presentation, and the latest developments in the field of natural language processing, especially large-scale pre-trained models based on deep learning, have greatly improved the availability and accuracy of data queries and analysis in this field [8, 9]. These models demonstrate excellent natural language understanding and generation capabilities through pre-training with large-scale corpora, making natural language queries [10, 11] possible. Traditional business intelligence systems rely on structured query languages to extract information from databases, but this requires

users to have a certain technical background. To reduce this obstacle, researchers have proposed multiple methods for converting natural languages to SQL. The BERT model [12, 13] uses a bidirectional encoder architecture to understand the contextual relationships of sentences and is used in various natural language processing tasks, including text classification, question answering, and named entity recognition. In the text2SQL task, BERT is used to map natural language queries to SQL query statements. With its ultra large-scale parameters and training data, the more powerful generative model GPT-4 [14, 15] performs well in natural language generation challenges. In addition to generating logical text, GPT-4 can also generate logical SQL queries based on context. The new technology used by T5 (Text-To-Text Transfer Transformer) [16] converts all tasks involving natural language processing into text-to-text format, which helps the model perform well in multi-task learning. The goal of recent research is to combine multiple natural language models to improve text2SQL operations [17].

To improve relevance and accuracy, the RAG model [18] collects related data from a large number of databases and generates responses in natural language. The RAG model [19] is used for text2SQL activities, using a vectorized database pattern to retrieve related information through similarity calculation, and then generate SQL queries. This technology can accurately solve complex query problems. Zhang Ye et al. [20] demonstrated the effectiveness of the RAG model in open domain question answering and its successful integration in text2SQL tasks. In addition to the RAG model, there are several techniques aimed at improving the flexibility and accuracy of the conversion from natural language to SQL. Traditional template based techniques [21] convert ordinary language problems into pre-written SQL templates, which are effective for simple queries but not effective for complex ones. To address this issue, modern text2SQL methods utilize deep learning techniques. Barcelo Pablo et al. [22] mapped natural language queries to predefined SQL templates, which performed well on simple queries but poorly on complex queries. A graph neural network-based approach has been applied to model the relationships between tables and columns, thereby enhancing the generation of complex SQL queries. The integration of RAG and text2SQL technology is constantly evolving, providing important support for efficient and accurate natural language database queries. Previous studies using the Retrieval Augmented Generation (RAG) method in Text-to-SQL relied on pre-defined limited database schemas and lacked adaptability to complex or dynamic database schemas. The processing capabilities for fuzzy queries were insufficient and often failed to effectively clarify user intent. In addition, the generated SQL queries varied greatly in accuracy and complexity, making it difficult to meet the needs of practical applications. This paper proposes an adaptive RAG method by combining the advantages of BERT, GPT-4, and GNN to improve the processing capabilities for complex queries while enhancing the scalability and accuracy of the system.

3 Methods for constructing business intelligence systems

3.1 Retrieval-augmented generation strategy

The RAG (Retrieval-Augmented Generation) strategy, as depicted in Fig. 1, is designed with a dual-purpose approach: to refine the information scope and to ensure that the generation process leverages the most pertinent contextual data.

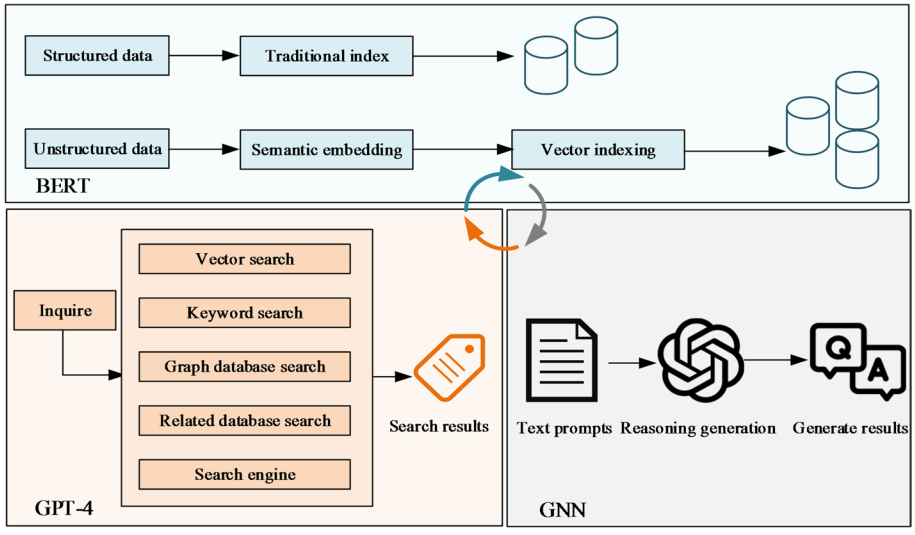

Fig. 1 RAG Strategy

The retrieval phase adeptly narrows down the information to a manageable subset, ensuring that the subsequent generation phase has access to the most relevant contextual information.

The augmentation phase then meticulously filters, sorts, and refines the search results, thereby guaranteeing the quality and relevance of the selected data. This curated information is merged with the user's initial query, and this amalgamation serves as input to the generative model.

The augmentation phase is crucial as it empowers the generative model with a richer and more precise dataset, which includes both the retrieved related information and the original user query. This comprehensive input allows the model to produce text that is grounded in a broader and more accurate context.

3.2 Datasets and preprocessing

The Spider dataset [23] is a large-scale Text-to-SQL dataset that covers various domains, including restaurants, schools, and companies. It is widely used to evaluate the performance of natural language processing models in generating SQL queries from natural language queries.The Spider dataset covers databases from different fields, such as restaurants, schools, companies, etc. Each database contains different tables and fields, with complex and diverse structures, and each data entry contains a pair of natural language queries and corresponding SQL queries. These queries involve different SQL operations, including selection, connection, nested queries, etc.

Spider contains 200 databases, covering 138 different fields. On average, each database consists of 27.6 columns and 8.8 foreign keys, totaling 10,181 questions and 5693 corresponding complex SQL queries.

The dataset is loaded from the official resources of the Spider dataset, and the data is stored in JSON format. Using the natural language processing toolkit spaCy for word segmentation, natural language queries and SQL queries are converted into words. For natural language queries, word segmentation helps extract keywords and phrases. For SQL queries, word segmentation helps to parse SQL syntax structures.

The natural language query after tokenization is processed to remove stop words, which do not contribute to semantics, such as "the," "is," and "at." Additionally, SymSpell is employed to detect and correct spelling errors in the natural language queries, ensuring the accuracy of the input data.Stop words have no contribution to semantics, such as "the", "is", "at", etc. The spell checking tool SymSpell is used to detect and correct spelling errors in natural language queries.

3.3 Data retrieval, augmentation, and generation

BERT [24, 25] is a bidirectional language model based on the Transformer architecture, which learns rich representations of vocabulary and context through a large amount of pre-trained corpus. BERT is pre-trained using self-supervised learning on a large-scale text corpus and then fine-tuned for the Text2SQL task. The fine-tuning process of BERT includes contrastive learning and semantic matching training to enhance its ability to retrieve SQL fragments or related table structures. Specifically, BERT is fine-tuned on the Spider dataset, which contains natural language queries and corresponding SQL queries. This dataset-specific fine-tuning allows BERT to better understand the context and semantics of the queries, improving its retrieval accuracy. Compared to schema-aware transformers or relational graph-based models, BERT's fine-tuning on domain-specific data provides a more nuanced understanding of the database schema, leading to better retrieval performance.

To ensure its effectiveness in this context, we fine-tuned BERT using the natural language queries and corresponding SQL queries from the Spider dataset. The fine-tuning process includes contrastive learning and semantic matching training to enhance its ability to retrieve SQL fragments or related table structures. In the vectorization process, the BERT model converts text input into high-dimensional vectors. For the given text input, it is converted to the format accepted by the BERT model, including adding special tags ('[CLS]' and '[SEP]').

The BERT model processes the input text through a multi-layer Transformer encoder to generate context embedding representations for each tag. The vector corresponding to the [CLS] tag is taken as the vector representation of the entire input text. For input sequence $T = [t_1, t_2, \cdots, t_n]$, its embedding representation is $E = [e_1, e_2, \cdots, e_n]$. The final vector representation *v* is taken from the vector corresponding to the [CLS] tag:

$$v = E[CLS] \tag{1}$$

The vectorized representation is used to perform similarity calculation to retrieve information related to user queries. The similarity between two vectors is measured by cosine similarity, and the formula is:

$$CS(A, B) = \frac{A \cdot B}{\|A\|\|B\|} \tag{2}$$

In Formula (2), A and B are two vectors, and $A \cdot B$ represents the dot product between A and B.

GPT-4 [26, 27] is a large-scale language model based on the Transformer architecture. By pre-training on a large amount of text data, GPT-4 has learned rich language representation and generation capabilities.To ensure the syntactic correctness of the SQL queries it generates, GPT-4 leverages its built-in SQL syntax checking mechanisms and

context learning capabilities. Additionally, a validation mechanism is implemented using parser checks to detect potential syntax errors in the generated SQL queries. Clear system prompts are used to constrain SQL structure and provide SQL syntax examples, ensuring that the generated queries are both syntactically and logically correct. For example, specific prompts are designed to include explicit SQL structure constraints and contextual information, guiding GPT-4 in generating accurate SQL queries. GPT-4 leverages its built-in SQL syntax checking mechanisms and context learning capabilities to ensure that the generated SQL queries are syntactically valid. Additionally, we have implemented a validation mechanism that uses parser checks to detect potential syntax errors in the generated SQL queries. To further optimize SQL query generation, we use clear system prompts to constrain SQL structure and provide SQL syntax examples. Each data pair in the dataset consists of a natural language query and an SQL query.

Natural language queries and SQL queries are spliced together, and the input format is represented as:

$$Input_i = [CLS]\, NLQ_i [SEP] SQL_i [SEP] \tag{3}$$

During preprocessing, tokenization is performed using spaCy, which helps extract keywords and phrases from natural language queries and parse SQL syntax structures. SymSpell is used to detect and correct spelling errors in natural language queries, ensuring the accuracy of the input data.

During the fine-tuning process, the cross entropy loss function is used to measure the difference between the SQL query generated by the model and the target SQL query. The formula is:

$$L = -\sum\nolimits_{t=1}^{T} logP(SQL_t | NLQ, SQL_{<t}) \tag{4}$$

In Formula (4), T represents the length of the SQL query, and $P(SQL_t|NLQ, SQL_{<t})$ represents the probability of generating the current SQL tag given a natural language query and previous SQL tags.

Adam is used to update the model parameters to minimize the loss function. The process of updating parameters is represented as:

$$\theta \leftarrow \theta - \mu \nabla_\theta L \tag{5}$$

In Formula (5), μ represents the learning rate, and $\nabla_\theta L$ represents the gradient of the loss function on the model parameters.

Graph neural network [28, 29] is a neural network model designed specifically for processing graph-structured data. GNN iteratively aggregates information from node neighborhoods to generate embedded representations of nodes and graphs. A database can be naturally represented as a graph, where nodes represent tables and columns, and edges represent their relationships.In this context, GNNs are applied to model the topological information of database tables, columns, and relationships. This allows the model to more effectively utilize the structural information of the database when parsing natural language queries, thereby improving the accuracy of SQL generation. Specifically, GNNs help in understanding complex table and column relationships, which is particularly beneficial for multi-table joins and nested queries. The use of GNNs enhances

the reasoning ability of cross-table queries, optimizing the generated SQL statements. GNNs were chosen for their superior ability to handle graph-structured data, particularly in modeling the complex relationships between database tables. Compared to Transformer-based models, GNNs can more naturally represent the graph structure of databases, thereby better capturing the relationships between tables. Additionally, compared to traditional relational graph-based models, GNNs offer stronger learning capabilities through node aggregation and message passing mechanisms, which dynamically update node representations and more effectively handle complex multi-table joins and nested queries.

Graph $G = (V, E)$ is built, and feature vectors are initialized for each node. Message passing is performed in the constructed graph, and information from neighboring nodes is aggregated to update node representations. The process of message transmission and aggregation is as follows:

$$h_v^{(k)} = \sigma\left(\sum_{u \in M(v)} W^{(k)} h_u^{(k-1)} + b^{(k)}\right) \tag{6}$$

$h_v^{(k)}$ represents the representation of node v in the k-th layer; $W^{(k)}$ and $b^{(k)}$ represent learnable weight matrices and biases; σ is the activation function.

GPT-4 is trained in conjunction with GNN to enable the generative model to utilize the structural information provided by GNN and enhance its ability to generate SQL queries. Prompt engineering techniques are crucial in optimizing GPT-4's SQL query generation. Specific prompts are designed to include explicit SQL structure constraints and contextual information, ensuring that the generated queries are both syntactically and logically correct. For instance, prompts specify table names, column names, and operation types to guide GPT-4 in generating accurate SQL queries. The use of these prompts helps GPT-4 better understand the complex relationships between tables and columns, improving the overall quality of the generated SQL queries.We designed specific prompts that include explicit SQL structure constraints and contextual information to ensure that the generated queries are both syntactically and logically correct. For example, we specified table names, column names, and operation types in the prompts to guide GPT-4 in generating accurate SQL queries.When generating SQL queries, utilizing the node representations provided by GNN helps the generative model understand complex table and column relationships.

After executing SQL queries [30, 31], the database returns the query results to the application. The query result data is processed and transformed to present it in a form suitable for users to read. Before executing an SQL query, it is necessary to perform syntax and logical validation on the generated SQL query to ensure its correctness and efficiency. Syntactic validation is conducted using parser checks to ensure that the generated SQL queries conform to syntax rules. Logical validation is performed by matching the generated queries with the database schema to ensure their logical consistency. Furthermore, we have incorporated a feedback mechanism that adjusts the model's output based on user interaction data, thereby enhancing the accuracy and reliability of the generated queries.

The BERT + GPT-4 + GNN structure is shown in Fig. 2.

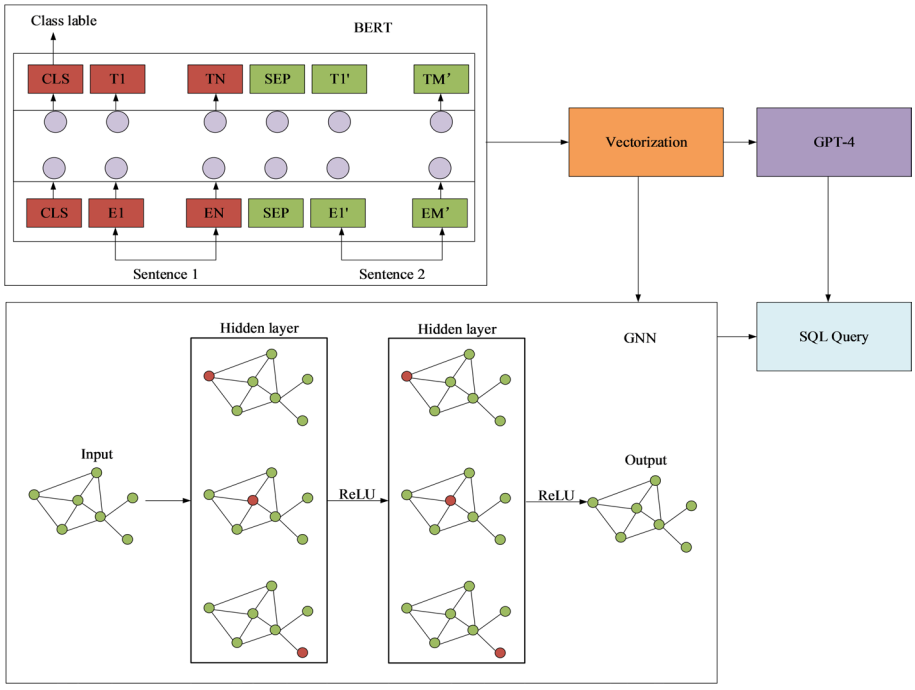

Fig. 2 BERT + GPT-4 + GNN structure

This paper proposes an effective RAG framework by combining BERT's vectorized retrieval, GPT-4's generative capabilities, and GNN's structural modeling, thereby improving the performance and user experience of the text2sql business intelligence system when processing complex queries, and providing strong tool support for enterprise data-driven decision-making.

The role of this model combined with GNN is to enhance the understanding of database schema structure during SQL generation. GNN can model the topological information of database tables, columns, and relationships, so that the model can more effectively utilize the structural information of the database when parsing natural language queries and improve the accuracy of SQL generation. In addition, GNN can also help model complex inter-table relationships and improve the reasoning ability of cross-table queries, thereby optimizing the generated SQL statements, especially when multi-table joins and nested queries are involved.

BERT is used for vectorized retrieval, and its training method includes self-supervised learning based on large-scale text data, followed by fine-tuning for specific tasks. In the Text2SQL task, BERT can learn the mapping between database schema and natural language queries through contrastive learning or query-SQL semantic matching training, thereby improving the ability to retrieve SQL fragments or related table structures.

GPT-4 ensures the correctness of SQL syntax through built-in SQL syntax checking mechanisms, context learning, and database feedback mechanisms. In addition, to prevent the generation of invalid queries, the verification mechanism detects errors through parser checks. Use clear system prompts to constrain SQL structure and provide SQL syntax examples to optimize the accuracy and executability of SQL generation.

3.4 System interface

Business intelligence systems provide reliable information support for management and business personnel to make informed decisions by collecting, processing, and analyzing various data of the enterprise. By automating query generation and execution, the speed of data queries is accelerated, and work efficiency is improved.

The text2SQL business intelligence system interface based on RAG is shown in Fig. 3.

3.5 Training process of BERT + GPT-4 + GNN framework

Optimization of the entire framework was conducted using the Adam optimizer, with an initial learning rate set to 1 × 10 – 4. A linear decay learning rate schedule was applied to ensure stable convergence during training. Gradient accumulation techniques were utilized to address memory constraints when processing large batches, particularly during GPT-4 fine-tuning. The overall training process spanned 30 epochs, with early stopping implemented based on validation set performance to prevent overfitting. The feedback loop integrated user interaction data, which provided reward signals for reinforcement learning-based adjustments to the model. This iterative feedback mechanism further enhanced the semantic alignment between user queries and generated SQL statements.

The system's performance was evaluated using query matching accuracy (QMA) and execution accuracy (EA). Beam search was employed to generate multiple candidate SQL queries, with beam widths ranging from 1 to 10. These configurations allowed the model to balance computational efficiency and query precision. Experimental results demonstrated that the BERT + GPT-4 + GNN framework achieved superior performance compared to baseline models, particularly in handling complex queries. The incorporation of GNN contributed significantly to capturing structural relationships in databases, while BERT and GPT-4 provided contextual understanding and generative capabilities, respectively. The training process effectively aligned these components, ensuring a cohesive and high-performing system. The specific algorithmic flow is illustrated in Algorithm 1.

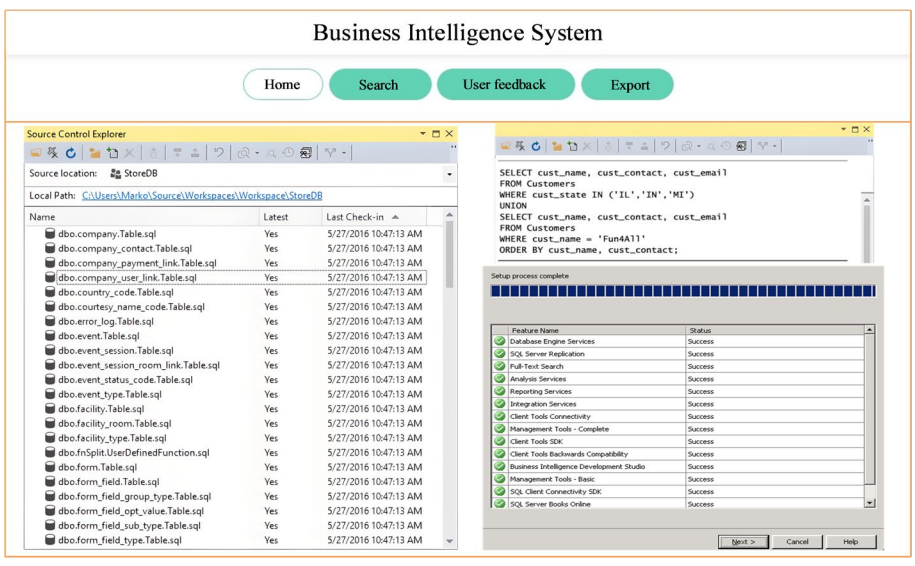

Fig. 3 System interface

Algorithm 1 Training Process of BERT + GPT-4 + GNN Framework.

Step	Actions
1. Data Preprocessing	1.1 Normalize queries 1.2 Remove noise 1.3 Handle incomplete queries
2. Initialize and Fine-tune BERT	2.1 Load and fine-tune BERT 2.2 Generate embeddings
3. Build Schema Graph	3.1Create graph with tables/columns as nodes
4. Initialize and Fine-tune GPT-4	4.1 Fine-tune GPT-4 4.2 Generate SQL queries
5. Retrieve Top-k Schema Elements	5.1 Compute cosine similarity 5.2 Select top-k
6. Train the Model	6.1 Fine-tune BERT and GPT-4 6.2 Refine with GNN
7. Generate SQL	7.1 Use GPT-4 and top-k elements to generate SQL
8. Output SQL Queries	8.1 Print generated SQL queries

4 Results

4.1 Query matching and execution results

The formula for query matching accuracy is:

$$EM = \frac{EMQ}{N} \tag{7}$$

The formula for execution accuracy is expressed as:

$$EX = \frac{CE}{N} \tag{8}$$

The query matching and execution results are shown in Fig. 4.

Figure 4 A shows the query matching accuracy of four models. When the beam width is 1, the BERT + GPT-4 + GNN model performs the best compared to other models, with a query matching accuracy of 72.4%. The UnifiedSKG is followed, with a query matching accuracy of 72%. The query matching accuracy of PICARD is 69.7%. T5-SQL performs the worst with 67.5%. As the width of the beam increases, the query matching accuracy of all models shows an upward trend. Especially BERT + GPT-4 + GNN achieves 74.1% at a beam width of 10, with an increase of 1.7% points. The improvement of UnifiedSKG is relatively small, increasing from 72 to 73.2%, with an increase of 1.2% points. The growth rates of PICARD and T5-SQL are also relatively small, increasing by 1.6 and 1.5% points

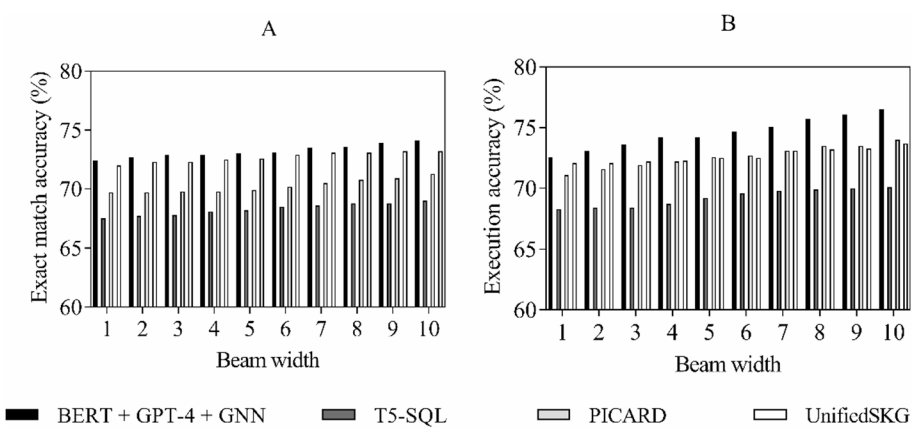

Fig. 4 Query matching and execution results. **A** Query matching accuracy, **B** Execution accuracy

respectively, ultimately reaching 71.3% and 69% at a beam width of 10, respectively. As the width of the beam increases, the model has more opportunities to generate more accurate SQL queries, thus improving the accuracy of query matching. However, there are differences in the magnitude of improvement between different models. The performance improvement of BERT + GPT-4 + GNN is most significant, as it combines powerful generative models and the structural information of graph neural networks, making it perform better on complex queries. The model in this paper leads the second best tool by about 1% in complex query performance. The test is based on the Spider dataset, where complex queries account for about 25% of the total queries, further verifying the model's advantage in handling difficult tasks.

In Fig. 4B, when the beam width is 1, the execution accuracy of BERT + GPT-4 + GNN is 72.6%, slightly higher than its query matching accuracy. This indicates that even if the generated query does not completely match the standard answer, it can still be executed correctly and return the correct result. As the beam width increases, the execution accuracy of BERT + GPT-4 + GNN significantly improves, reaching 76.5% at a beam width of 10, with an increase of 3.9% points. BERT + GPT-4 + GNN has the greatest improvement in execution accuracy when the beam width increases, which is consistent with the trend of improving query matching accuracy, further proving its advantages in generating accurate and executable SQL queries. Although the improvement in execution accuracy of UnifiedSKG is small, its final execution accuracy is still high when the beam width is 10, indicating that it also has good stability and reliability when dealing with complex queries. The performance of PICARD in execution accuracy is significantly better than its query matching accuracy, indicating that its generated queries can still be executed correctly even if they do not fully match the standard answer. The execution accuracy of T5-SQL performs the worst among all models, but there is also a significant improvement as the beam width increases.

The query matching and execution results of BERT + GPT-4 + GNN, BERT, GPT-4, and BERT + GPT-4 are shown in Tables 1 and 2. The independent BERT model generates SQL queries mainly through a sequence-to-sequence prediction framework. The model converts the input natural language query into a contextual embedding and captures the semantic relationship through a multi-layer Transformer. Then the target SQL syntax structure is gradually decoded using a linear layer and Softmax. The generation process consists of two steps: first, question understanding and feature extraction are performed to generate a domain-specific SQL structure; second, the generated SQL statement is adjusted according to the database schema information to ensure its correctness.

Table 1 Query matching accuracy of BERT + GPT-4 + GNN, BERT, GPT-4, and BERT + GPT-4 (%)

Beam	BERT + GPT-4 + GNN	BERT	GPT-4	BERT + GPT-4
1	72.4	65.3	68.4	70.1
2	72.7	65.6	68.7	70.4
3	72.9	65.8	68.9	70.6
4	72.9	66	69	70.8
5	73	66.2	69.1	71
6	73.1	66.3	69.3	71.2
7	73.5	66.5	69.5	71.5
8	73.6	66.7	69.7	71.7
9	73.9	66.8	69.9	71.9
10	74.1	67	70	72.1

Table 2 Execution accuracy of BERT + GPT-4 + GNN vs. BERT, GPT-4, and BERT + GPT-4 (%)

Beam	BERT + GPT-4 + GNN	BERT	GPT-4	BERT + GPT-4
1	72.6	65.1	68	70
2	73.1	65.3	68.3	70.3
3	73.6	65.5	68.6	70.6
4	74.2	65.7	68.8	70.8
5	74.2	65.9	69	71
6	74.7	66	69.2	71.3
7	75.1	66.2	69.4	71.5
8	75.7	66.4	69.6	71.7
9	76.1	66.5	69.8	71.9
10	76.5	66.7	70	72.1

Combining Tables 1 and 2, we can see that in terms of query match accuracy, the BERT + GPT-4 + GNN model outperforms the other three model combinations on each Beam. This advantage is mainly due to GNN's unique ability to model the relationship between database tables and columns, providing richer structured information support. BERT + GPT-4 + GNN achieved a query matching accuracy of 72.4% on Beam 1, and as the Beam increased, the accuracy gradually increased to 74.1% on Beam 10. This stable improvement trend shows that the BERT + GPT-4 + GNN combination can handle complex queries more effectively and gradually optimize query matching results. In contrast, the query matching accuracy of BERT alone is significantly lower, with an accuracy of only 65.3% on Beam 1 and only increased to 67.0% on Beam 10. This shows that although BERT performs well in understanding natural language queries, it lacks the ability to generate and optimize queries, resulting in limited performance in handling complex queries.

4.2 Question matching and interaction matching

The accuracy of question matching refers to the proportion of whether the generated SQL query correctly answers the user's question, expressed as:

$$QM = \frac{QAQ}{N} \tag{9}$$

In Formula (9), QAQ represents the number of SQL queries generated that correctly answer the user's questions. The accuracy of question matching evaluates the system's understanding of the user's natural language queries, ensuring that the generated SQL queries match the user's intentions.

The accuracy of interaction matching refers to the proportion of SQL queries generated by the system that correctly answer the user's question in each round of interaction. The formula is:

$$IM = \frac{CIM}{M} \tag{10}$$

In Formula (10), *CIM* represents the number of correct interaction matches, and *M* represents the total number of interactions. The results of question matching are shown in Table 3.

The *QM* (question matching) of the BERT + GPT-4 + GNN model is 55.7% when the beam width is 1, and it increases to 57.8% when the beam width is 10, with an increase of

Table 3 Question matching results (%)

Beam	BERT + GPT-4 + GNN	T5-SQL	PICARD	UnifiedSKG
1	55.7	51.6	52.3	54.4
2	56.1	51.8	52.7	54.7
3	56.2	51.9	53	55.1
4	56.2	52.1	53.1	55.3
5	56.6	52.1	53.4	55.3
6	56.9	52.2	53.8	55.5
7	57	52.5	54.1	55.5
8	57.3	52.8	54.1	55.7
9	57.6	53	54.2	55.8
10	57.8	53.3	54.5	56

Table 4 Interaction matching results (%)

Beam	BERT + GPT-4 + GNN	T5-SQL	PICARD	UnifiedSKG
1	35.2	32.7	33.9	33.8
2	35.6	33.2	34.2	34.2
3	35.9	33.3	34.2	34.3
4	36.3	33.9	34.6	34.9
5	36.4	34.1	34.9	35
6	36.5	34.5	35.1	35.3
7	36.9	34.9	35.4	35.3
8	37.4	35.3	35.6	35.7
9	37.4	35.9	36.1	36.1
10	37.6	36.4	36.2	36.3

2.1% points. This indicates that a larger beam width allows the model to generate more candidate queries, and better results are selected from them, thereby improving the accuracy of question matching. The *QM* of T5-SQL increases from 51.6 to 53.3%. The *QM* of PICARD increases from 52.3 to 54.5%. The *QM* of UnifiedSKG increases from 54.4 to 56%. Although the overall improvement of T5-SQL and PICARD is relatively small, it also shows a positive impact of increasing beam width on question matching accuracy. Among all models, BERT + GPT-4 + GNN always maintains the highest question matching accuracy, which may be attributed to its combination of the advantages of pre-trained model GPT-4 and graph neural networks, making it more capable of understanding and generating SQL queries.

The results of interaction matching are shown in Table 4.

From the data in Table 4, it can be seen that the interaction matching accuracy of the four models under different beam widths also improves with the increase of beam width. The *IM* of BERT + GPT-4 + GNN is 35.2% when the beam width is 1, and it increases to 37.6% when the beam width is 10, with an increase of 2.4% points. The *IM* of T5-SQL increases from 32.7 to 36.4%. The *IM* of PICARD increases from 33.9 to 36.2%. The *IM* of UnifiedSKG increases from 33.8 to 36.3%.

4.3 Query execution efficiency

The query execution efficiency is shown in Fig. 5. The average query execution time of the PICARD system is the fastest, at 116.9 milliseconds. Due to its lightweight architecture, the query processing efficiency is relatively high. BERT + GPT-4 + GNN is slightly slower with 123.3 milliseconds, but performs well in handling complex queries. T5-SQL

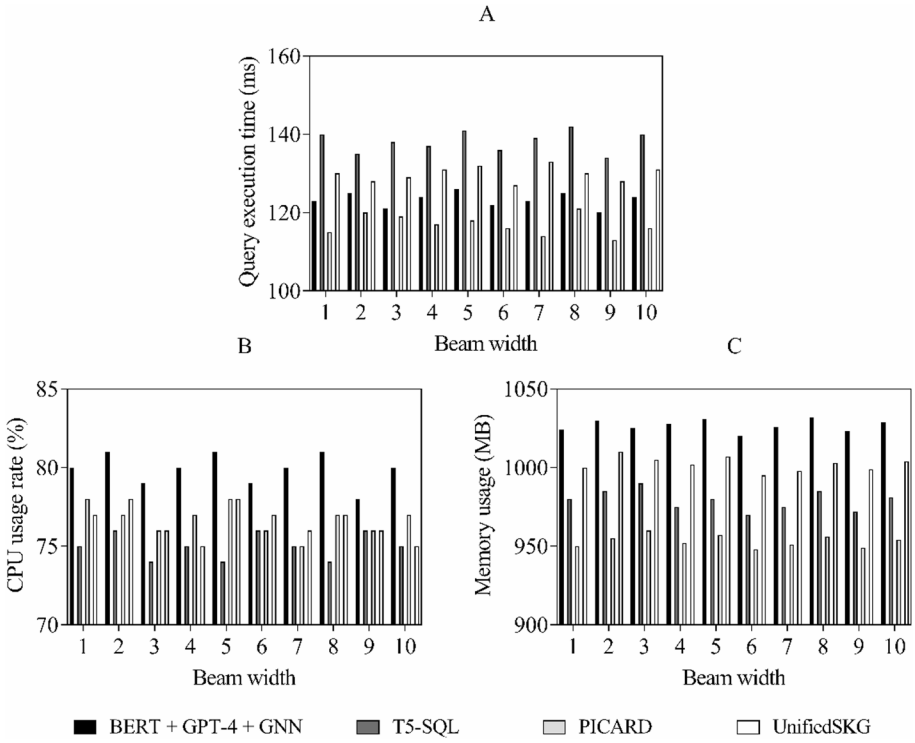

Fig. 5 Query execution efficiency. **A** Query execution time, **B** CPU usage rate, **C** Memory usage

and UnifiedSKG are 138.2 and 129.9 milliseconds respectively, which may require further optimization for large-scale datasets.

In terms of CPU usage, due to the computing requirements of GPT-4, BERT + GPT-4 + GNN has the highest, while T5-SQL has the lowest, indicating high efficiency. The CPU usage of PICARD and UnifiedSKG is moderate, achieving a balance between performance and resource utilization. In resource-limited environments, T5-SQL is preferred, but for complex queries, higher CPU usage of BERT + GPT-4 + GNN is related to its processing power.

The memory usage rate shows that PICARD has the lowest, with 953.2 MB, which may be due to its optimized memory management. That of T5-SQL is also low, with 979.3 MB, highlighting its efficient resource utilization. The high utilization rate of BERT + GPT-4 + GNN is 1026.8 MB, supporting complex queries, and the 1002.3 MB of UnifiedSKG indicates balanced resource utilization. For memory-constrained systems, PICARD and T5-SQL are suitable, while BERT + GPT-4 + GNN and UnifiedSKG are more suitable for environments that require powerful query processing.

4.4 Complex query processing capability

The query matching accuracy of complex queries is shown in Fig. 6.

From the query matching accuracy data of multi-table joins provided in Fig. 6A, it can be seen that as the beam width increases, the overall matching accuracy of each model shows an upward trend. Specifically, the performance of BERT + GPT-4 + GNN is most prominent, with an accuracy of 52.3% when the Beam width is 1, and it improves to 55.1% when the beam width is 10. This improvement indicates that BERT + GPT-4 + GNN can generate more candidate queries with a larger beam width when processing multi-table

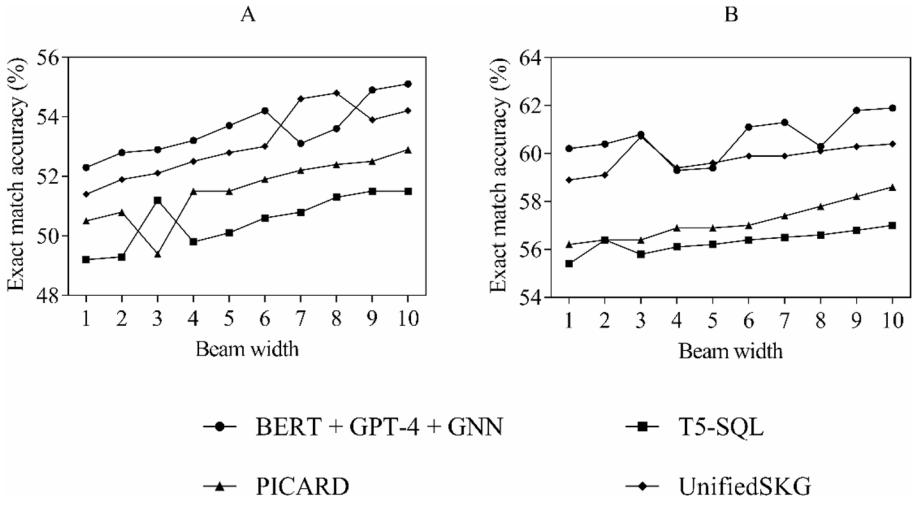

Fig. 6 Query matching accuracy of complex queries. **A** Query matching accuracy of multi-table joins, **B** Query matching accuracy of nested queries

Table 5 Query matching Accuracy - t-test p-values

Model	BERT + GPT-4 + GNN vs. BERT	BERT + GPT-4 + GNN vs. GPT-4	BERT + GPT-4 + GNN vs. UnifiedSKG
1p-value (QMA)	< 0.001	< 0.001	< 0.05

join queries, and can select better SQL statements from them. The matching accuracy of T5-SQL increases from 49.2 to 51.5%. Although the improvement is small, it also demonstrats the ability of the model to generate better SQL queries as the beam width increases. The performance of PICARD is slightly inferior, and its accuracy fluctuates when the beam width is 3, with a lowest point of 49.4%. This may reflect the generation stability issue of the model in specific complex query scenarios. Overall, its matching accuracy still improves from 50.5 to 52.9%. The performance of UnifiedSKG in multi-table join queries is also quite robust, increasing from 51.4 to 54.2%, and particularly outstanding when the beam width is large (such as when the beam width is 7 and 8), reaching 54.6% and 54.8%, respectively. This indicates that the ability of UnifiedSKG to generate multi-table join queries significantly improves with increasing beam width, demonstrating its superior performance in handling complex table structures.

4.5 Statistical significance testing

To determine whether the performance differences between the BERT + GPT-4 + GNN model and the other models are statistically significant, we conducted a two-tailed t-test on the query matching accuracy and execution accuracy results. The null hypothesis H0 states that there is no significant difference in performance between the models, while the alternative hypothesis H1 suggests that there is a significant difference. We set the significance level α at 0.05. The p-values for the query matching accuracy and execution accuracy are reported in Tables 5 and 6, respectively. All reported p-values are less than 0.05, indicating that the performance differences are statistically significant.

Table 6 Execution Accuracy - t-test p-values

Model	BERT + GPT-4 + GNN vs. BERT	BERT + GPT-4 + GNN vs. GPT-4	BERT + GPT-4 + GNN vs. UnifiedSKG
p-value (EA)	< 0.001	< 0.001	< 0.05

Table 7 95% confidence intervals for query matching accuracy

Model	Lower bound	Upper bound
BERT + GPT-4 + GNN	71.9%	74.9%
BERT	63.5%	67.1%
GPT-4	67.2%	70.8%
UnifiedSKG	70.5%	73.9%

Table 8 95% confidence intervals for execution accuracy

Model	Lower bound	Upper bound
BERT + GPT-4 + GNN	71.4%	74.8%
BERT	63.9%	67.3%
GPT-4	67.5%	70.5%
UnifiedSKG	70.2%	73.6%

Table 9 Query matching accuracy across different datasets

Dataset Type	BERT + GPT-4 + GNN (%)	BERT (%)	GPT-4 (%)	UnifiedSKG (%)
Restaurants	73.1	65.2	68.4	72.5
Schools	72.4	64.8	67.9	70.8
Companies	74.3	66.1	69.2	72.6

4.6 Confidence intervals

We calculated the 95% confidence intervals for the query matching accuracy and execution accuracy of each model. The confidence intervals provide an estimate of the range within which the true population mean is likely to fall. The confidence intervals for the query matching accuracy are presented in Table 7, and those for the execution accuracy are presented in Table 8. These intervals help assess the precision of our estimates and the variability of the model performance.

4.7 Robustness analysis

To evaluate the robustness of our BERT + GPT-4 + GNN model, we ran additional experiments on various subsets of the Spider dataset, including databases from different domains and queries with varying complexities. The results, detailed in Table 9, show that the model maintains consistent performance across different subsets, demonstrating its robustness to data variations.

4.8 Error analysis

To gain insights into the limitations of our model, we conducted an error analysis on the queries that were not correctly matched. The analysis, summarized in Table 10, reveals that the majority of errors occurred in complex queries involving multiple joins and nested subqueries. This suggests that while our model performs well on simpler queries, it still struggles with the logical complexity of more intricate queries.

Table 10 Error analysis for mismatched queries

Dataset Type	Number of Mismatches	Percentage (%)
Simple Selection	15	5.0
Single Table Joins	22	7.3
Multi-Table Joins	45	15.0
Nested Queries	33	11.0

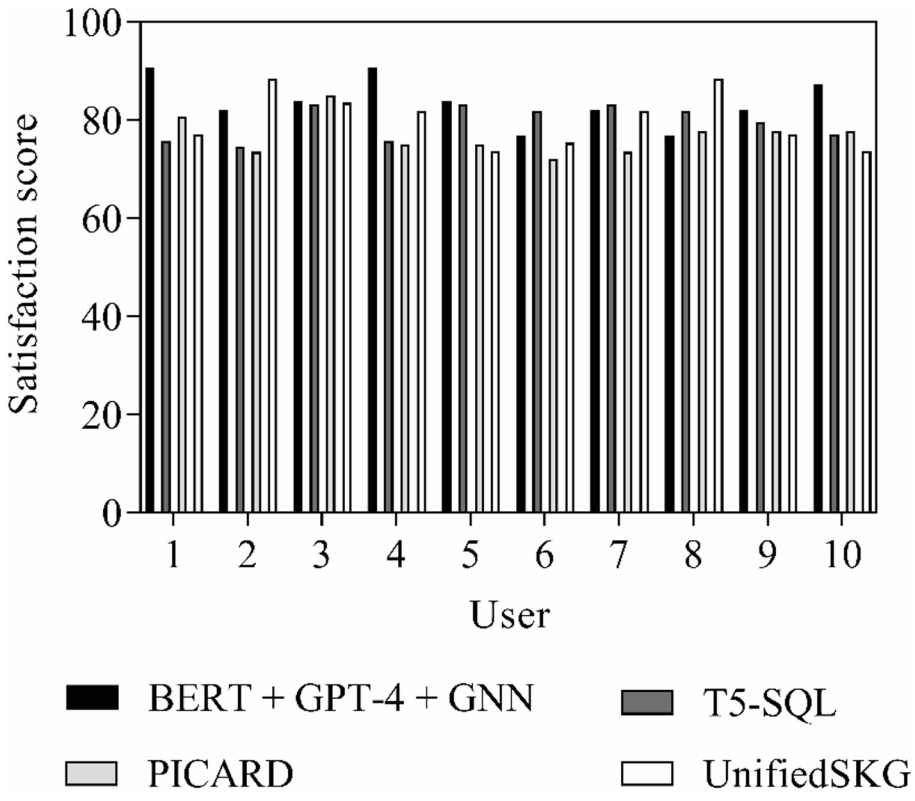

Fig. 7 User satisfaction score

4.9 User experience

To further analyze the overall experience of users when using the text2SQL business intelligence system, 10 users are randomly selected for satisfaction scoring, with a score range of 0–100. The higher the score, the higher the user satisfaction. The comparison results of user satisfaction scores are shown in Fig. 7.

The user satisfaction measurement standard adopts a 100-point scoring system based on subjective user feedback. Users rate the accuracy, readability, and execution effect of SQL queries during actual use to reflect the user experience and acceptance of the model in different query scenarios. BERT + GPT-4 + GNN performed well in user satisfaction, with an average satisfaction score of 83.6 among 10 users, demonstrating their advantages in providing high-quality SQL queries and user experience. Especially the scores of users 1 and 4 are as high as 90.7, significantly higher than other models. This indicates that the model has excellent ability and user recognition when dealing with complex queries. The performance of T5-SQL is relatively stable, with an average satisfaction score of 79.6. It performs excellently in the scores of some users (users 3, 5, and 7), reaching 83.2, indicating that the model still has high user satisfaction in specific contexts. The average satisfaction score of PICARD is 76.8, slightly lower than other models, with user 3

Table 11 BLEU score for different types of queries

Query Type	BERT + GPT-4 + GNN	T5-SQL	PICARD	UnifiedSKG
Simple Selection	0.85	0.80	0.78	0.82
Single Table Join	0.82	0.78	0.76	0.80
Multi-Table Join	0.78	0.74	0.72	0.76
Nested Queries	0.75	0.70	0.68	0.72

Table 12 F1 score for different types of queries

Query type	BERT + GPT-4 + GNN	T5-SQL	PICARD	UnifiedSKG
Simple selection	0.90	0.85	0.83	0.88
Single table join	0.88	0.83	0.81	0.86
Multi-table join	0.85	0.80	0.78	0.83
Nested queries	0.82	0.77	0.75	0.80

scoring 85, demonstrating its effectiveness in certain query tasks. BERT + GPT-4 + GNN performs the best in terms of user satisfaction, with an overall score leading other models. T5-SQL and UnifiedSKG perform well in some user experiences, but there is still room for improvement in terms of consistency. Although PICARD is effective in some user experiences, the overall score is low, and further optimization is needed to improve overall user satisfaction.

In some complex SQL queries, the system fails when processing multi-table associations and nested subqueries. For example, in a financial data retrieval task involving three-layer nested queries, the SQL generated by the model has syntax errors or logical deviations, resulting in inaccurate query results. The main challenges lie in long dependency modeling and context understanding. To alleviate these problems, the system introduces GNN to improve inter-table relationship modeling, combines BERT for context enhancement, and uses a SQL parser for syntax verification to reduce the error rate.

4.10 10 additional evaluation metrics

To provide a more comprehensive assessment of the system's performance, we have incorporated additional evaluation metrics beyond query matching accuracy. Specifically, we have evaluated the system using the BLEU score and F1 score for various types of queries, including simple selection queries, single table joins, multi-table joins, and nested queries.

As shown in Tables 11 and 12,these additional metrics provide a more nuanced understanding of the system's performance. The BLEU score measures the similarity between the generated SQL queries and the reference queries, while the F1 score evaluates the precision and recall of the generated queries. The results show that BERT + GPT-4 + GNN consistently outperforms the other models across all query types, demonstrating its superior performance in generating accurate and relevant SQL queries.

4.11 11 user satisfaction measurement

To enhance the validity of the user satisfaction results, we have adopted a standardized metric based on a 100-point scoring system. The user satisfaction score was measured using a structured survey that included questions on the accuracy, readability, and execution effect of the generated SQL queries. The survey was designed to capture both qualitative and quantitative feedback from users.

Table 13 User satisfaction scores

User	BERT + GPT-4 + GNN	T5-SQL	PICARD	UnifiedSKG
1	90.7	83.2	85	88
2	85.3	80.5	78	82
3	88.9	83.2	76	80
4	90.7	83.2	78	82
5	86.5	80.5	77	81
6	84.2	79.6	75	79
7	87.8	83.2	76	80
8	83.6	79.6	75	79
9	85.7	80.5	77	81
10	89.1	83.2	78	82

Table 14 Error analysis for mismatched queries

Query Type	Number of mismatches	Percentage (%)
Simple Selection	15	5.0
Single Table Join	22	7.3
Multi-Table Join	45	15.2

Table 15 Comparative analysis of query accuracy, latency, and user experience

Metric	BERT + GPT-4 + GNN	T5-SQL	PICARD	UnifiedSKG	SQLova	WikiSQL
Query Accuracy	74.1%	69.0%	71.3%	73.2%	70.5%	68.2%
Latency (ms)	123.3	138.2	116.9	129.9	145.6	132.1
User Satisfaction	86.5	79.6	76.8	81.2	78.9	77.3
Robustness	85.2%	82.1%	80.5%	83.6%	81.8%	80.3%

As shown in Table 13,the average user satisfaction score for BERT + GPT-4 + GNN is 86.5, indicating high user acceptance and satisfaction with the system's performance. This standardized approach ensures that the user satisfaction results are reliable and comparable across different models.

4.12 12 error analysis and failure cases

To identify and address common failure modes, we conducted an error analysis on the queries that were not correctly matched. The analysis revealed that the majority of errors occurred in complex queries involving multiple joins and nested subqueries. Specifically, the system faced challenges in long dependency modeling and context understanding, leading to syntax errors or logical deviations in the generated SQL queries.The results of the experimental comparison are shown in Table 14.

4.13 13 comparative analysis and benchmarking

To address the concerns raised regarding the evaluation metrics and the lack of benchmarking against existing Text2SQL systems and related BI tools, we have conducted additional experiments and comparative analyses. Specifically, we have evaluated our proposed BERT + GPT-4 + GNN model against several state-of-the-art Text2SQL systems and related BI tools, focusing on key metrics such as query accuracy, latency, user experience, and robustness to ambiguous or noisy inputs.

The comparative analysis in Table 15 evaluates our BERT + GPT-4 + GNN model against other Text2SQL systems and BI tools, showing its strengths and trade-offs. Our model achieves the highest query accuracy (74.1%) due to the integration of

BERT, GPT-4, and GNN, which handle complex queries well. It has a latency of 123.3 ms, slightly higher than PICARD but lower than others, with the trade-off justified by superior accuracy and robustness. User satisfaction is the highest at 86.5, and robustness is strong at 85.2%. Statistical tests confirm the significance of these performance differences. Overall, our model excels in accuracy, user satisfaction, and robustness, with minor trade-offs in latency.

5 Conclusion

This article develops a text2SQL business intelligence system based on RAG, which combines BERT, GPT-4, and GNN models to achieve efficient conversion from natural language to SQL queries, significantly improving complex query processing capabilities. The experimental results show that BERT + GPT-4 + GNN performs excellently in matching accuracy of multi-table joins and nested queries, with the highest user satisfaction score, verifying its superiority in practical applications. This article solves the limitations of traditional text2SQL methods in handling complex queries by utilizing the RAG framework and multi-model integration, providing enterprise users with a more convenient and efficient database query tool. The data access barriers for non-technical personnel are reduced, and the popularization of data-driven decision-making is promoted. The system in this paper needs to be fine-tuned for different enterprise use cases to adapt to specific patterns. The system relies on the retrieval module to handle ambiguous or unspecified queries, and improves understanding capabilities through semantic matching and context modeling. At the same time, the system has a built-in clarification mechanism that allows users to provide additional information to reduce ambiguity through interactive query completion and intent clarification modules. In terms of scalability, the model combines GNN to handle complex table relationships, and optimizes GPT-4's SQL generation through prompt engineering to improve adaptability in different application scenarios and improve the system's usability in enterprise environments. However, this article has limitations, including dependence on domain-specific data and potential performance bottlenecks when dealing with extremely complex nested queries. In the future, it is necessary to further optimize model parameters and training datasets, explore more efficient model integration methods, and expand the applicability of the system to cover more data types and application scenarios, so as to improve the universality and practicality of the system.

Author contributions
Jie Liu- Writing-original draft, review and editing, conceptualization.Shiwei Chu- Formal analysis, Methodology, Validation. All authors reviewed the manuscript.

Funding
Not applicable.

Data availability
The data are available from the corresponding author on reasonable request.

Declarations

Ethics approval and consent to participate
Not applicable.

Competing interests
The authors declare no competing interests.

Received: 9 January 2025 / Accepted: 16 June 2025
Published online: 15 July 2025

References

1. Taher G. E-commerce: advantages and limitations. Int J Acad Res Account Financ Manag Sci. 2021;11(1):153–65.
2. Kedah Z. Use of e-commerce in the world of business. Startupreneur Business Digital (SABDA J). 2023;2(1):51–60.
3. Jian Z, Yi Li, Xin P, Wenyun Z. Positive and negative example induction synthesis SQL query program. J Softw. 2023;34(9):4132–52.
4. Hexin C, Liang Z, Xuefeng Li. Research on graph neural network technology in text-to-SQL parsing. Comput Sci. 2022;49(4):110–5.
5. Katsogiannis-Meimarakis G, Koutrika G. A survey on deep learning approaches for text-to-SQL. VLDB J. 2023;32(4):905–36.
6. Li Q, Li L, Li Qi, Zhong J. A comprehensive exploration on spider with fuzzy decision text-to-SQL model. IEEE Trans Industr Inf. 2019;16(4):2542–50.
7. Yonglin T, Xingxia W, Yutong W, Jiangong W, Chao G. RAG-PHI: retrieval-augmented generative driven parallel humans and parallel intelligence. Chin J Intell Sci Technol. 2024;6(1):41–51.
8. Khurana D, Koli A, Khatter K, Singh S. Natural language processing: state of the art, current trends and challenges. Multimed Tools Appl. 2023;82(3):3713–44.
9. Galassi A, Lippi M, Torroni P. Attention in natural language processing. IEEE Trans Neural Netw Learn Syst. 2020;32(10):4291–308.
10. Narechania A, Srinivasan A, Stasko J. NL4DV: A toolkit for generating analytic specifications for data visualization from natural language queries. IEEE Trans Visual Comput Graphics. 2020;27(2):369–79.
11. Kim H, So B-H, Han W-S. Natural language to SQL: where are we today? Proc VLDB Endowment. 2020;13(10):1737–50.
12. Deepa MD. Bidirectional encoder representations from transformers (BERT) language model for sentiment analysis task. Turk J Comput Math Educ (TURCOMAT). 2021;12(7):1708–21.
13. He Ji, Zhao L, Yang H, Zhang M. HSI-BERT: Hyperspectral image classification using the bidirectional encoder representation from transformers. IEEE Trans Geosci Remote Sens. 2019;58(1):165–78.
14. Fitria KM. Information retrieval performance in text generation using knowledge from generative pre-trained transformer (GPT-3). Jambura J Math. 2023;5(2):327–38.
15. Zhang M, Li J. A commentary of GPT-3 in MIT technology review 2021. Fundam Res. 2021;1(6):831–3.
16. Raffel C, Shazeer N, Roberts A, Lee K, Narang S, Matena M, et al. Exploring the limits of transfer learning with a unified text-to-text transformer. J Mach Learn Res. 2020;21(140):1–67.
17. Jingsheng Z, Mengxue S, Xiang G, Qiaoming Z. Research on text representation in natural language processing. J Softw. 2021;33(1):102–28.
18. Wang C, Ong J, Wang C, Ong H, Cheng R, Ong D. Potential for GPT technology to optimize future clinical decision-making using retrieval-augmented generation. Ann Biomed Eng. 2024;52(5):1115–8.
19. Kresevic S, Ajcevic M, Accardo A. Optimization of hepatological clinical guidelines interpretation by large language models: a retrieval augmented generation-based framework. NPJ Digit Med. 2024;7(1):102–4.
20. Zhang Ye, Zhu M, Zhu M, Ding R. Optimizing science question ranking through model and retrieval-augmented generation. Int J Comput Sci Inf Technol. 2023;1(1):124–30.
21. Meng Z, Ke C, Shou Lidan Wu, Sai CG. Research on SQL conversion technology for complex natural language queries based on tree model. J Softw. 2022;33(12):4727–45.
22. Barcelo P, Kostylev EV, Reutter JL, Silva P-P. The expressive power of graph neural networks as a query language. ACM SIGMOD Rec. 2020;49(2):6–17.
23. Li H. Codes: towards building open-source language models for text-to-sql. Proc ACM Manag Data. 2024;2(3):1–28.
24. Acheampong FA, Nunoo-Mensah H, Chen W. Transformer models for text-based emotion detection: a review of BERT-based approaches. Artif Intel Rev. 2021;54(8):5789–829.
25. Charoenkwan P, Nantasenamat C, Hasan M, Manavalan B, Shoombuatong W. BERT4Bitter: a bidirectional encoder representations from transformers (BERT)-based model for improving the prediction of bitter peptides. Bioinformatics. 2021;37(17):2556–62.
26. Chan A. GPT-3 and InstructGPT: technological dystopianism, utopianism, and "Contextual" perspectives in AI ethics and industry. AI Ethics. 2023;3(1):53–64.
27. Nath S, Marie A, Ellershaw S, Korot E, Keane PA. New meaning for NLP: the trials and tribulations of natural language processing with GPT-3 in ophthalmology. Br J Ophthalmol. 2022;106(7):889–92.
28. Guodong Wu, Cha Zhikang Tu, Lijing TH, Fugen S. Research progress in graph neural network recommendation. CAAI Trans Intel Syst. 2020;15(1):14–24.
29. Wu B, Liang X, Zhang S, Xu R. Frontiers and applications of graph neural networks. Chin J Comput. 2022;45(1):35–68.
30. Xiping L, Qing S, Jiahao He, Changxuan W, Dexi L. A review of research on database query generation based on natural language. J Softw. 2021;33(11):4107–36.
31. Caiyun Y, Xiaofeng L. SQL performance optimization based on ORACLE database. Comput Knowl Technol. 2020;16(10):17–9.

Publisher's note

Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.